TestCube vs LLMDroid: how to read the result

The short version: on the same 13 apps, the two tools are roughly even. TestCube's averages look a bit higher, but that gap is not statistically solid. What is clear is when each tool wins, and that TestCube gets similar coverage with fewer taps.

Feature coverage in this comparison is weighted coverage: the mean share of gold steps that actually happened (mean of each feature's completion ratio). It is not 'fully covered features / total', which is near zero for both tools.

Final means (n = 13)

Metric	TestCube	LLMDroid	Difference	95% CI	Per-app wins
Code coverage	19.43%	18.09%	+1.33 pp	-0.98 to +3.65	8-5
Feature coverage (weighted)	15.58%	12.04%	+3.54 pp	-0.32 to +7.40	7-6
Actions per run	352	594	-41%	-	-
Code coverage per 100 actions	5.52	3.05	+81%	-	-

Both confidence intervals cross zero. Do not report these means as a demonstrated win. The efficiency ratio (fewer actions for similar coverage) is the number that holds up.

Per-app figure

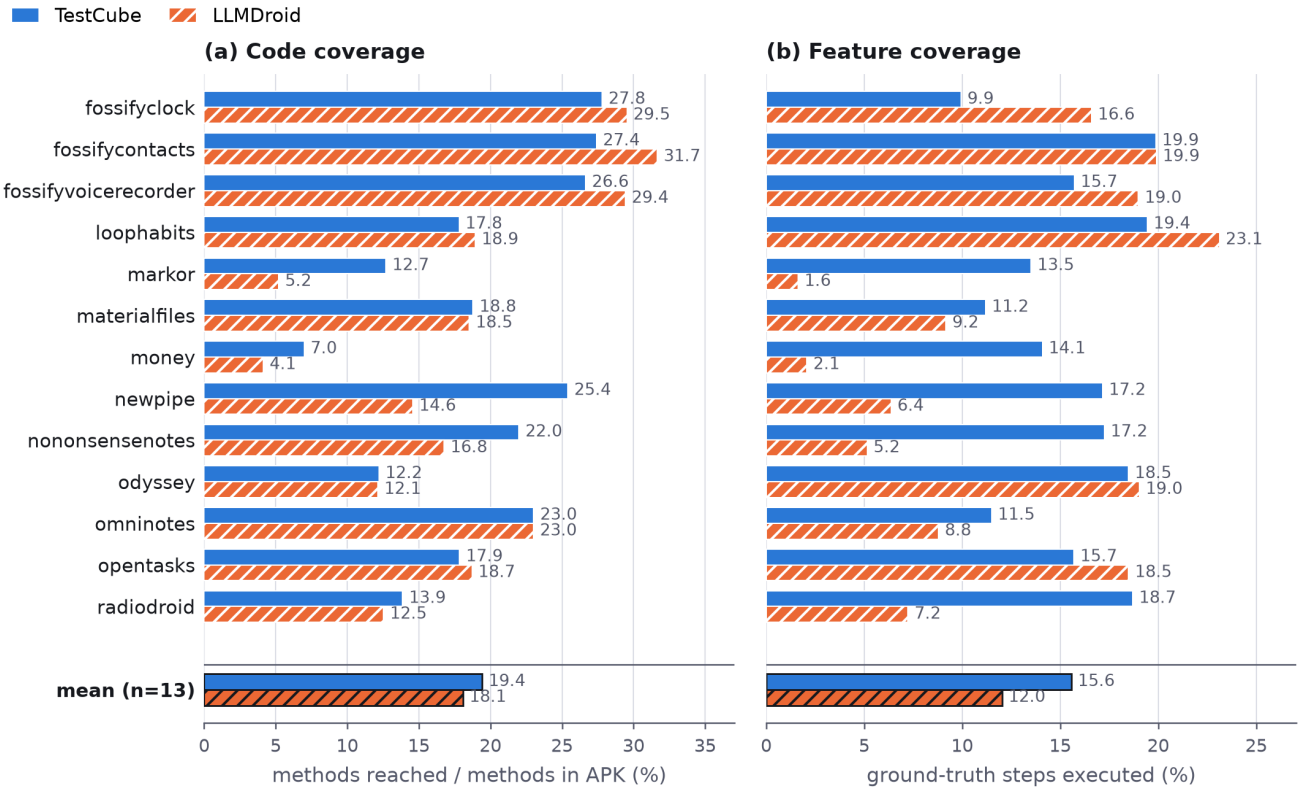


Figure: (a) code coverage from AndroLog probes; (b) weighted feature coverage. Blue = TestCube, hatched orange = LLMDroid. Mean row at the bottom.

When TestCube is better

TestCube wins when LLMDroid's exploration falls apart - not when the app is simply 'harder'. Typical pattern: a first-run wall, permissions, login, or a fragile LLM loop. LLMDroid never really gets into the app, or it issues only a handful of actions and stops. TestCube still has a feature list, so it keeps trying those steps.

App	Feature % (TC / LD)	Why TestCube leads
markor	13.51 / 1.63	LLMDroid ran many actions but never left the permission screen.
money	14.09 / 2.08	LLMDroid issued very few actions (~138). TestCube kept the feature list.
newpipe	17.16 / 6.37	LLMDroid issued ~130 actions. Guided search still progressed.
radiodroid	18.70 / 7.24	LLMDroid stalled at ~90 actions after the LLM stopped.
nononsensenotes	17.25 / 5.15	LLMDroid explored a lot but did not complete gold steps.
omninotes	11.50 / 8.79	Smaller edge: guidance beat unfocused wandering.
materialfiles	11.19 / 9.17	Smaller edge; same pattern.

Paper sentence: TestCube is stronger when the tester must stay on a specified capability after setup, dialogs, or a stalled LLM - not because it always covers more code.

When LLMDroid won

LLMDroid wins on small, shallow apps where coverage-guided wandering just works, and it is allowed to tap much more often. Typical pattern: few screens, easy navigation, no hard login wall. LLMDroid issued about 700+ actions; TestCube issued about 300. More wandering covers more of a small app. Margins are small (-0.04 to -6.7 pp on features), except Fossify Clock.

App	Feature % (TC / LD)	Why LLMDroid leads
fossifyclock	9.94 / 16.60	Small app; LLMDroid issued 729 actions vs TestCube 216.
fossifycontacts	19.87 / 19.91	Essentially a tie; LLMDroid slightly more exploration.
fossifyvoicerecorder	15.72 / 18.99	High-volume wandering on a shallow app.
loophabits	19.44 / 23.12	Easy navigation; ~747 vs 309 actions.
opentasks	15.70 / 18.49	Same: throughput, not a setup trap.
odyssey	18.48 / 19.04	Near tie; LLMDroid's extra actions barely move the needle.

Paper sentence: LLMDroid is stronger when cheap, high-volume exploration is enough - small apps, no setup trap, coverage-driven wandering.

How to portray this in the paper

Do not write 'TestCube beats LLMDroid'. Reviewers will look at the confidence intervals (both cross zero) and the degraded LLMDroid runs (markor, radiodroid).

1. Headline: comparable means, mixed per-app outcomes.
On paired 1-hour runs, mean code and weighted feature coverage are comparable. Per-app wins are mixed (code 8-5, feature 7-6).

2. Let the figure show heterogeneity.
LLMDroid on Fossify; TestCube on markor / money / newpipe. That split is why the mean interval is wide. Put it in the caption.

3. The claim that survives is efficiency.
TestCube reaches that coverage with 352 actions vs 594 (~41% fewer). Frame as efficiency and attribution, not as higher coverage.

4. Interpret, do not hide.
TestCube wins on robustness to stalled exploration; LLMDroid wins on throughput on simple apps. That matches the method: TestCube pays for one-feature-at-a-time plus model calls; LLMDroid pays for more cheap DFS until coverage plateaus.

5. Name the degraded baseline runs.
Markor and RadioDroid LLMDroid runs were degraded (never entered the app / LLM stalled at 90 actions). Either exclude them in a sensitivity row or say so in the caption. Keeping them is a real robustness result, but not a fair 'same healthy run' comparison.

6. Name the feature metric correctly.
Call it weighted feature coverage (step completion ratio). Do not imply strict covered/total.

Paragraph you can paste

Under a one-hour budget on 13 shared apps, TestCube and LLMDroid achieve similar mean code coverage (19.4% vs 18.1%) and weighted feature coverage (15.6% vs 12.0%); neither paired difference is statistically significant. Outcomes split by application: LLMDroid leads on small, easily explored apps (Fossify suite, Loop Habit Tracker, OpenTasks), where it issues roughly twice as many actions. TestCube leads when LLMDroid's exploration stalls in setup or issues few actions (Markor, Money Manager, NewPipe, RadioDroid, NoNonsense Notes). TestCube attains comparable coverage with 41% fewer actions (352 vs 594), consistent with feature-guided search rather than coverage-maximizing wandering.

Suggested figure caption

Per-app coverage for TestCube and LLMDroid over a one-hour budget on identical instrumented APKs. (a) Code coverage from AndroLog bytecode probes; (b) weighted feature coverage (mean completion ratio of ground-truth steps). TestCube means: 19.4% vs 18.1% code; 15.6% vs 12.0% feature. Neither paired difference is significant (code +1.33 pp, 95% CI [-0.98, +3.65]; feature +3.54 pp, 95% CI [-0.32, +7.40]). TestCube used 352 actions per run against LLMDroid's 594.

One-line takeaway: not a knockout - a split by app type, plus efficiency.